CS434a/541a: Pattern Recognition
Prof. Olga Veksler

Lecture 7

Today

- Problems of high dimensional data, “the curse of dimensionality”
  - running time
  - overfitting
  - number of samples required
- Dimensionality Reduction Methods
  - Principle Component Analysis (today)

Curse of Dimensionality: Complexity

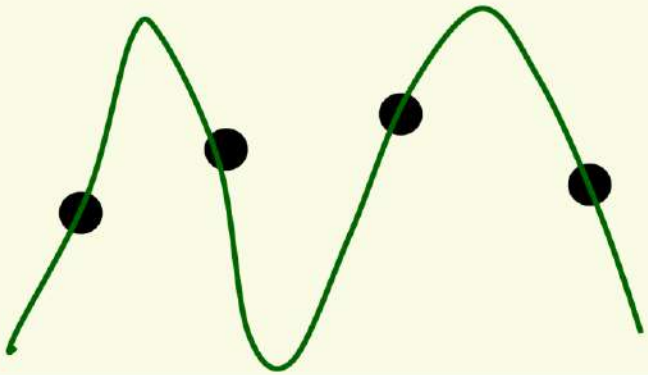
- Complexity (running time) increases with dimension ***d***
- A lot of methods have at least O(***nd***^{2}) complexity, where ***n*** is the number of samples
  - For example if we need to estimate covariance matrix
- So as ***d*** becomes large, O(***nd***^{2}) complexity may be too costly

Curse of Dimensionality: Overfitting

- If ***d*** is large, ***n***, the number of samples, may be too small for accurate parameter estimation
- For example, covariance matrix has d^2 parameters:

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \cdots & \sigma_{1d} \\ \vdots & \ddots & \vdots \\ \sigma_{d1} & \cdots & \sigma_d^2 \end{bmatrix}$$

- For accurate estimation, ***n*** should be much bigger than d^2
- Otherwise model is too complicated for the data, ***overfitting***:

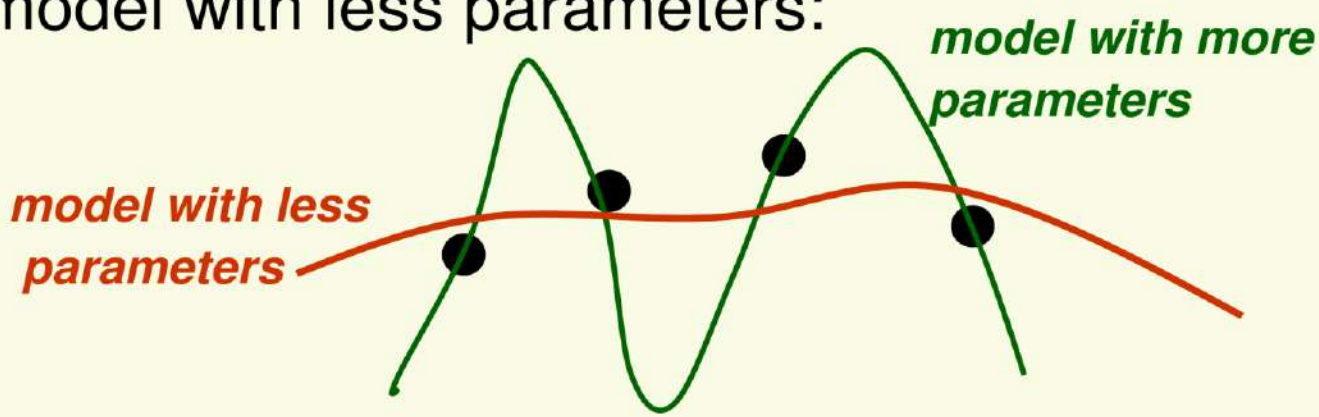


Curse of Dimensionality: Overfitting

- Paradox: If $n < d^2$ we are better off assuming that features are uncorrelated, even if we know this assumption is wrong
- In this case, the covariance matrix has only d parameters:

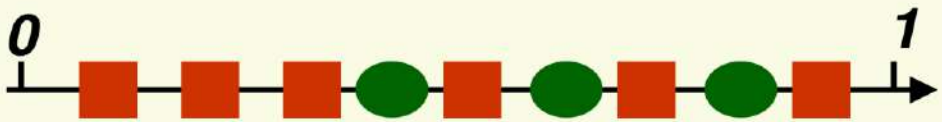
$$\Sigma = \begin{bmatrix} \sigma_1^2 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_d^2 \end{bmatrix}$$

- We are likely to avoid overfitting because we fit a model with less parameters:



Curse of Dimensionality: Number of Samples

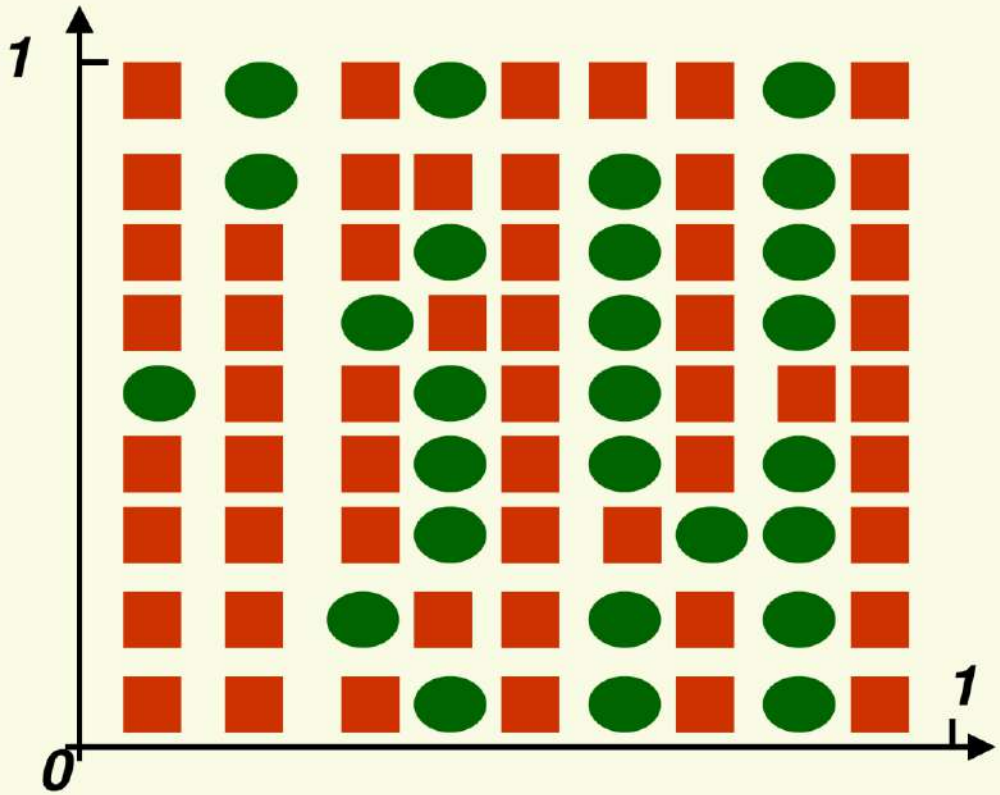
- Suppose we want to use the nearest neighbor approach with ***k*** = 1 (***1NN***)
- Suppose we start with only one feature



- This feature is not discriminative, i.e. it does not separate the classes well
- We decide to use 2 features. For the 1NN method to work well, need a lot of samples, i.e. samples have to be dense
- To maintain the same density as in 1D (9 samples per unit length), how many samples do we need?

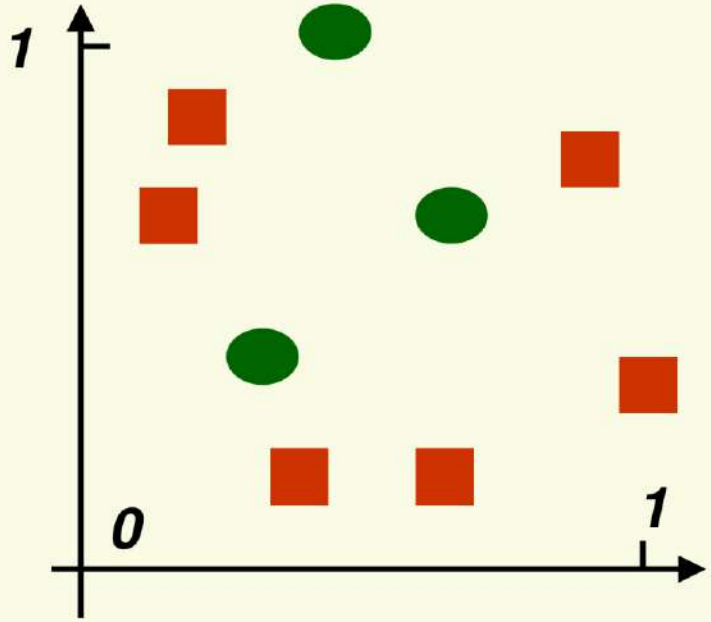
Curse of Dimensionality: Number of Samples

- We need 9^2 samples to maintain the same density as in *1D*



Curse of Dimensionality: Number of Samples

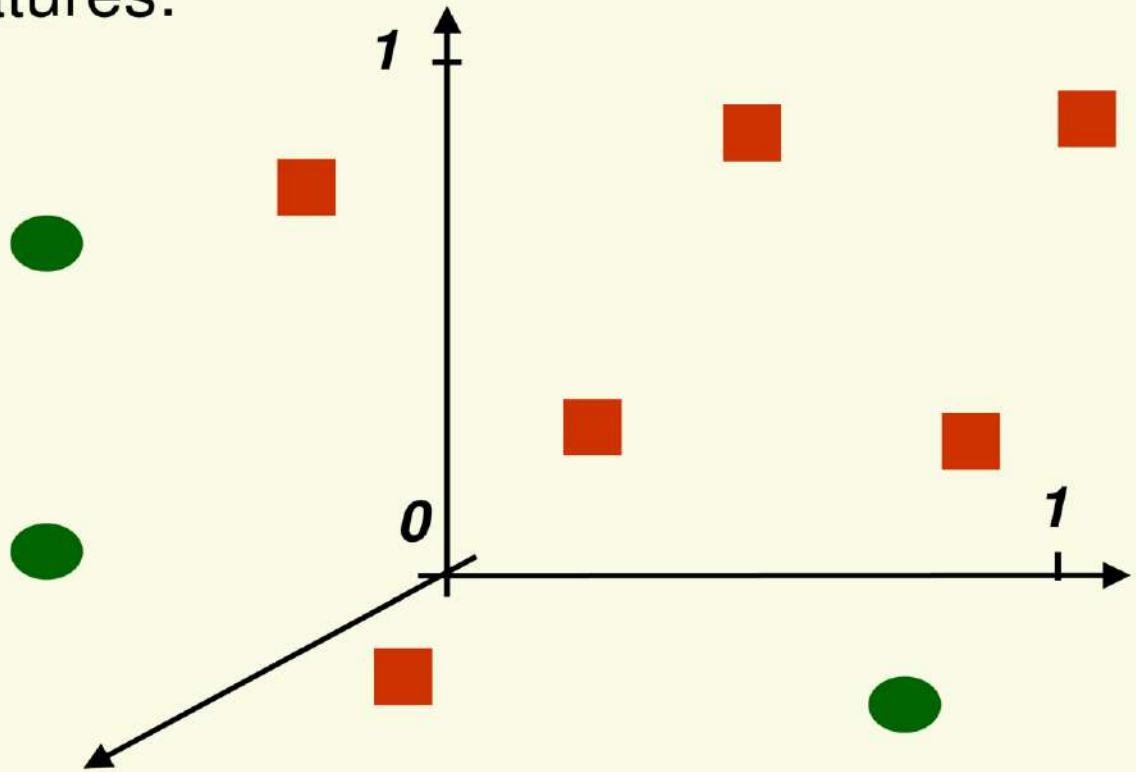
- Of course, when we go from 1 feature to 2, no one gives us more samples, we still have 9



- This is way too sparse for ***1NN*** to work well

Curse of Dimensionality: Number of Samples

- Things go from bad to worse if we decide to use 3 features:



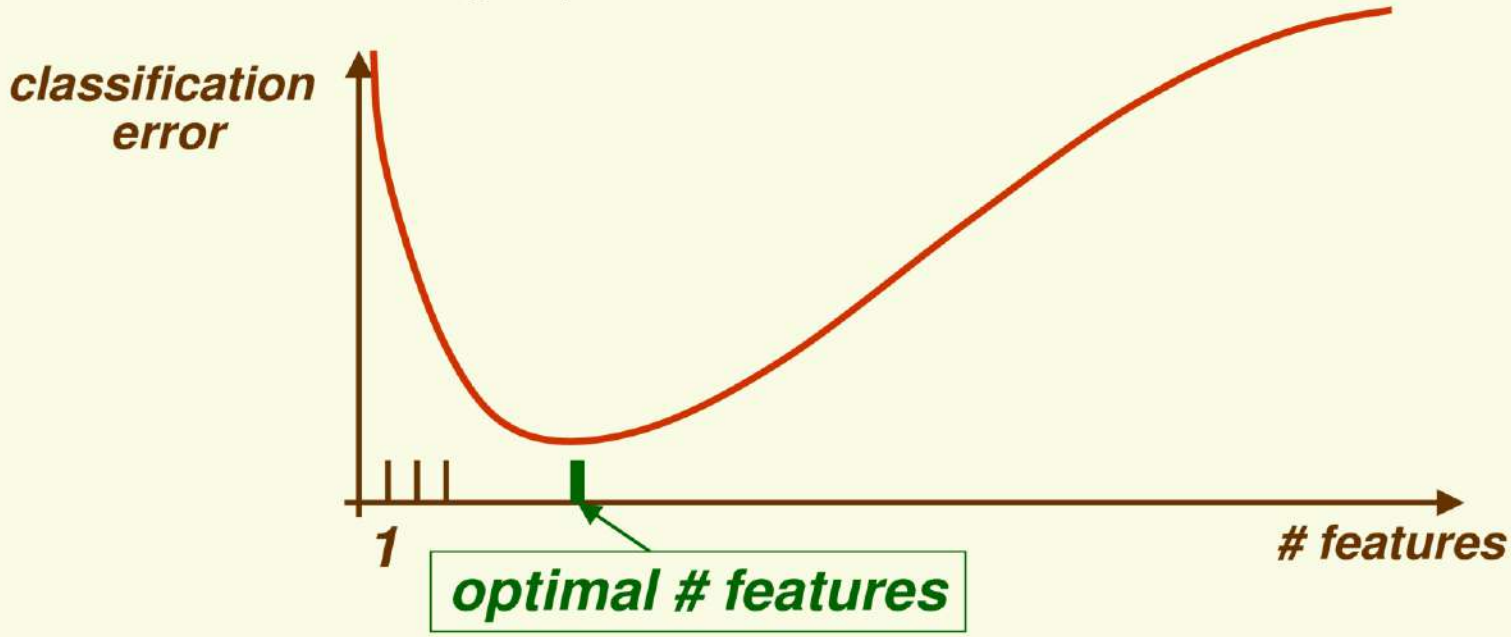
- If ***9*** was dense enough in 1D, in 3D we need $9^3 = 729$ samples!

Curse of Dimensionality: Number of Samples

- In general, if ***n*** samples is dense enough in ***1D***
- Then in ***d*** dimensions we need n^d samples!
- And n^d grows really really fast as a function of ***d***
- Common pitfall:
  - If we can't solve a problem with a few features, adding more features seems like a good idea
  - However the number of samples usually stays the same
  - The method with more features is likely to perform worse instead of expected better

Curse of Dimensionality: Number of Samples

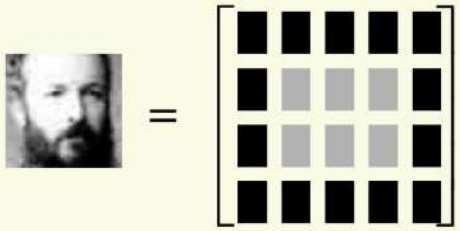
- For a fixed number of samples, as we add features, the graph of classification error:



- Thus for each fixed sample size ***n***, there is the optimal number of features to use

The Curse of Dimensionality

- We should try to avoid creating lot of features
- Often no choice, problem starts with many features
- Example: Face Detection
  - One sample point is ***k*** by ***m*** array of pixels



  - Feature extraction is not trivial, usually every pixel is taken as a feature
  - Typical dimension is 20 by 20 = 400
  - Suppose ***10*** samples are dense enough for 1 dimension. Need only $\boldsymbol{10^{400}}$ samples

Dimensionality Reduction

- High dimensionality is challenging and redundant
- It is natural to try to reduce dimensionality
- Reduce dimensionality by feature combination: combine old features $\boldsymbol{x}$ to create new features $\boldsymbol{y}$

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \rightarrow \boldsymbol{f}\left(\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}\right) = \begin{bmatrix} y_1 \\ \vdots \\ y_k \end{bmatrix} = \boldsymbol{y} \quad \textit{with } k < d$$

- For example,

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \rightarrow \begin{bmatrix} x_1 + x_2 \\ x_3 + x_4 \end{bmatrix} = \boldsymbol{y}$$

- Ideally, the new vector $\boldsymbol{y}$ should retain from $\boldsymbol{x}$ all information important for classification

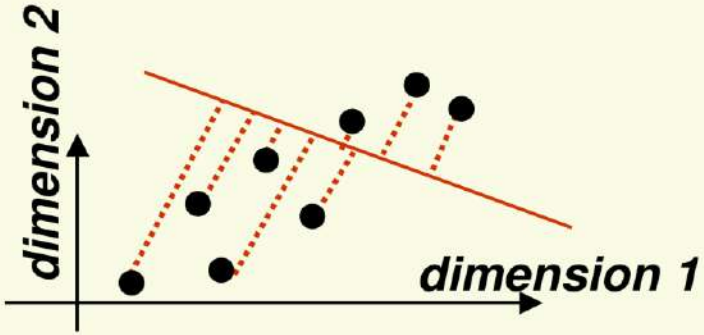
Dimensionality Reduction

- The best $\boldsymbol{f}(\boldsymbol{x})$ is most likely a non-linear function
- Linear functions are easier to find though
- For now, assume that $\boldsymbol{f}(\boldsymbol{x})$ is a linear mapping
- Thus it can be represented by a matrix $\boldsymbol{W}$:

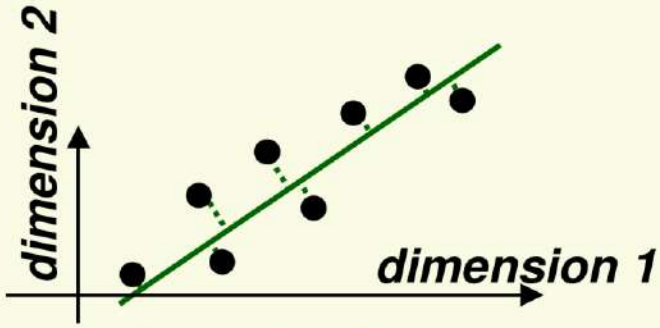
$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \Rightarrow W \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} = \begin{bmatrix} w_{11} & \cdots & w_{1d} \\ \vdots & & \vdots \\ w_{k1} & \cdots & w_{kd} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_k \end{bmatrix} \quad \textit{with } k < d$$

Principle Component Analysis (PCA)

- Main idea: seek most accurate data representation in a lower dimensional space
- Example in 2-D
  - Project data to 1-D subspace (a line) which minimize the projection error



large projection errors, bad line to project to

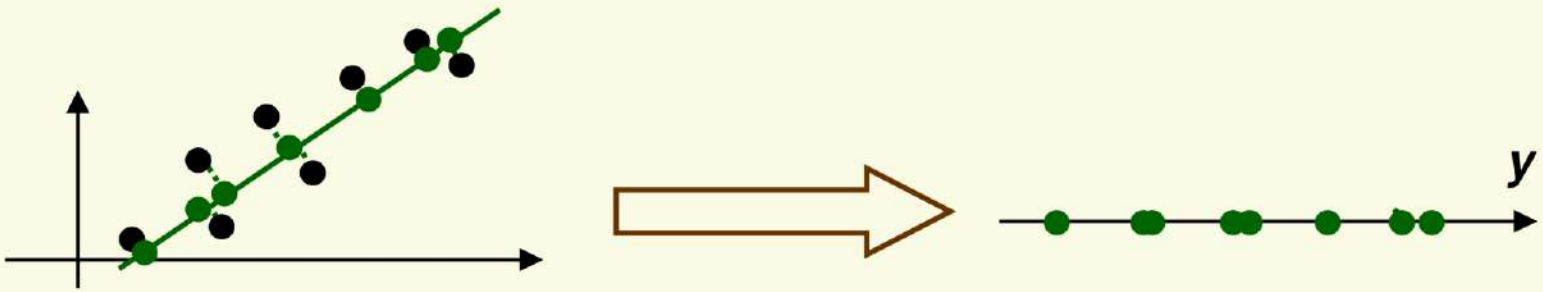


small projection errors, good line to project to

- Notice that the the good line to use for projection lies in the direction of largest variance

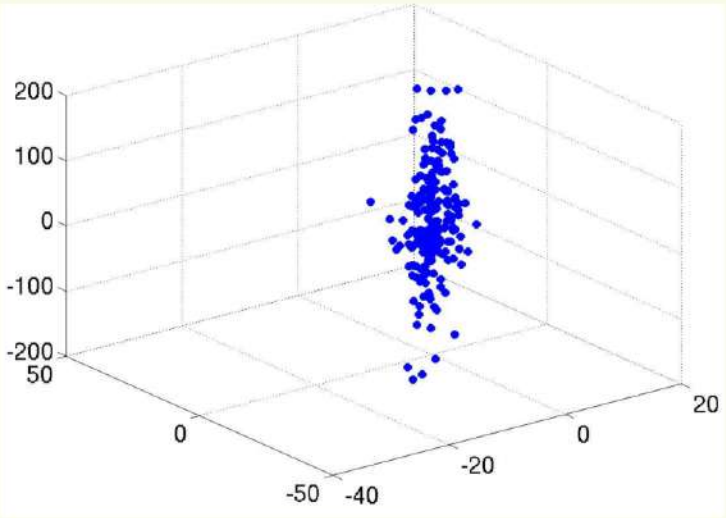
PCA

- After the data is projected on the best line, need to transform the coordinate system to get 1D representation for vector ***y***

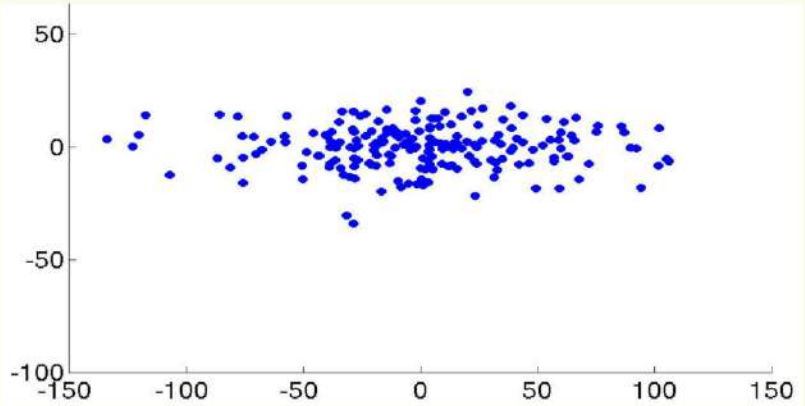


- Note that new data ***y*** has the same variance as old data ***x*** in the direction of the green line
- PCA preserves largest variances in the data. We will prove this statement, for now it is just an intuition of what PCA will do

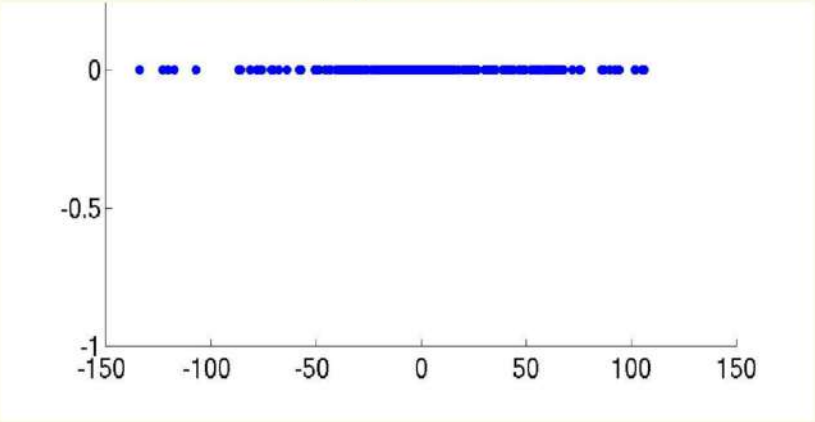
PCA: Approximation of Elliptical Cloud in 3D



best 2D approximation

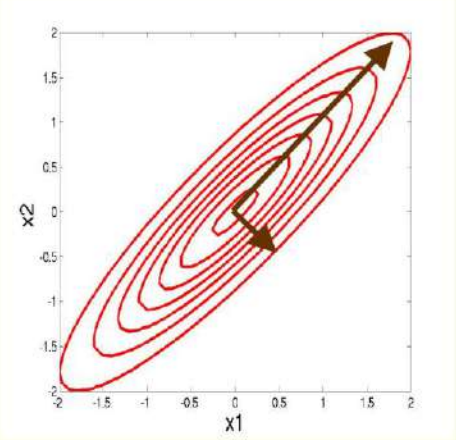


best 1D approximation



PCA

- What is the direction of largest variance in data?
- Recall that if $\boldsymbol{x}$ has multivariate distribution $N(\mu,\Sigma)$, direction of largest variance is given by eigenvector corresponding to the largest eigenvalue of Σ

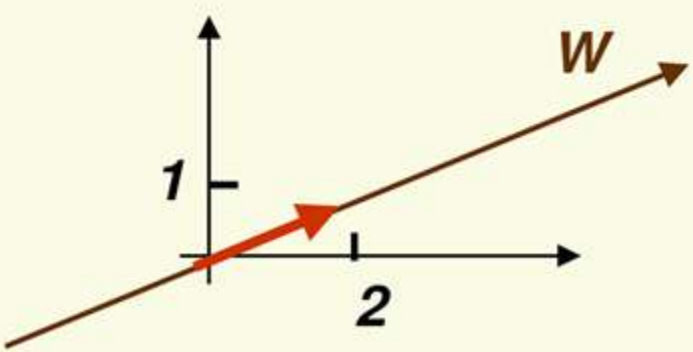


- This is a hint that we should be looking at the covariance matrix of the data (note that PCA can be applied to distributions other than Gaussian)

PCA: Linear Algebra for Derivation

- Let $\boldsymbol{V}$ be a $\boldsymbol{d}$ dimensional linear space, and $\boldsymbol{W}$ be a $\boldsymbol{k}$ dimensional linear subspace of $\boldsymbol{V}$
- We can always find a set of $\boldsymbol{d}$ dimensional vectors $\{\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_k\}$ which forms an orthonormal basis for $\boldsymbol{W}$
  - $\langle \boldsymbol{e}_i, \boldsymbol{e}_j \rangle = 0$ if $\boldsymbol{i}$ is not equal to $\boldsymbol{j}$ and $\langle \boldsymbol{e}_i, \boldsymbol{e}_i \rangle = 1$
- Thus any vector in $\boldsymbol{W}$ can be written as

$$\alpha_1 \mathbf{e}_1 + \alpha_2 \mathbf{e}_2 + \ldots + \alpha_k \mathbf{e}_k = \sum_{i=1}^{k} \alpha_i \mathbf{e}_i \quad \textbf{for scalars} \quad \alpha_1, \ldots, \alpha_k$$

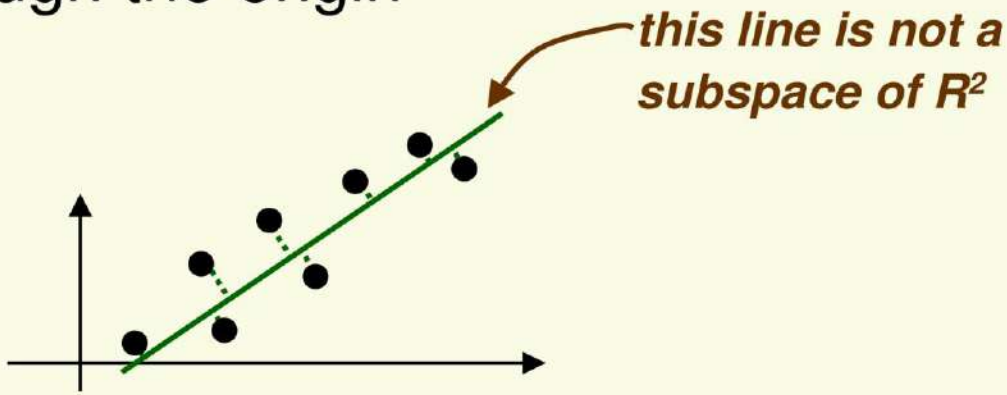


Let $\boldsymbol{V} = \boldsymbol{R}^2$ and $\boldsymbol{W}$ be the line x-2y=0. Then the orthonormal basis for W is

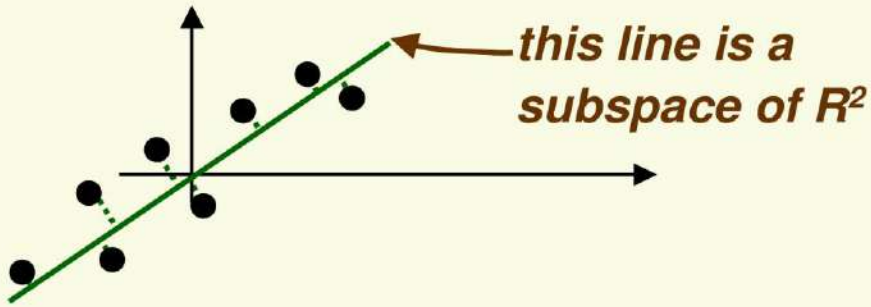
$$\left\{ \begin{bmatrix} 1/\sqrt{5} \\ 2/\sqrt{5} \end{bmatrix} \right\}$$

PCA: Linear Algebra for Derivation

- Recall that subspace ***W*** contains the zero vector, i.e. it goes through the origin



- For derivation, it will be convenient to project to subspace ***W***: thus we need to shift everything

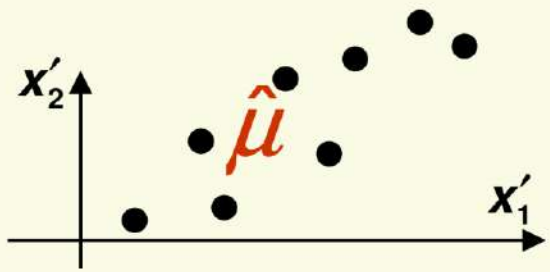


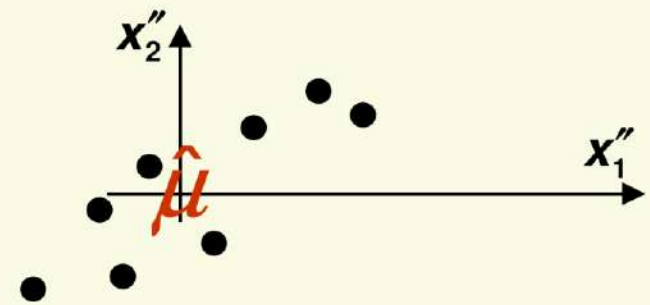
PCA Derivation: Shift by the Mean Vector

- Before PCA, subtract sample mean from the data

$$\boldsymbol{x} - \frac{1}{n}\sum_{i=1}^{n}\boldsymbol{x}_i = \boldsymbol{x} - \hat{\mu}$$

- The new data has zero mean: $E(\boldsymbol{X}-E(\boldsymbol{X})) = E(\boldsymbol{X})-E(\boldsymbol{X}) = 0$
- All we did is change the coordinate system



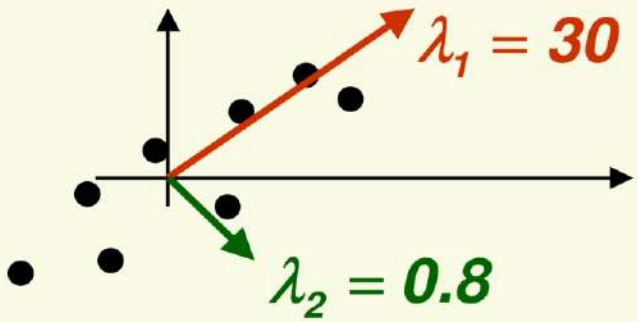


- Another way to look at it:
  - first step of getting $\boldsymbol{y}$ is to subtract the mean of $\boldsymbol{x}$

$$\boldsymbol{x} \rightarrow \boldsymbol{y} = \boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{g}(\boldsymbol{x} - \hat{\mu})$$

PCA

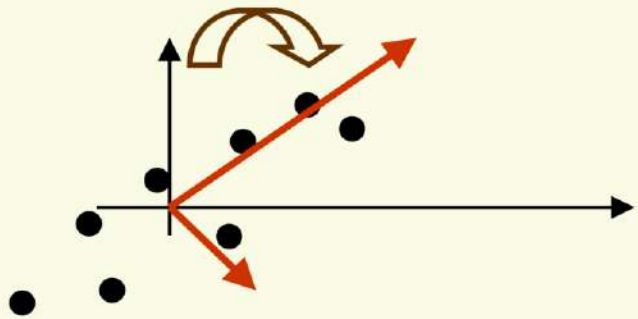
- The larger the eigenvalue of ***S***, the larger is the variance in the direction of corresponding eigenvector



- This result is exactly what we expected: project ***x*** into subspace of dimension ***k*** which has the largest variance
- This is very intuitive: restrict attention to directions where the scatter is the greatest

PCA

- Thus PCA can be thought of as finding new orthogonal basis by rotating the old axis until the directions of maximum variance are found



PCA as Data Approximation

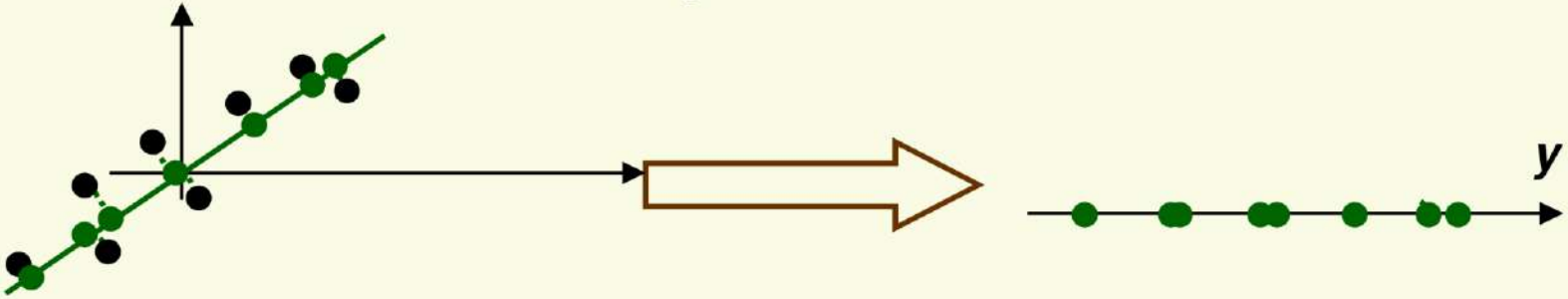
- Let $\{\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_d\}$ be all $\boldsymbol{d}$ eigenvectors of the scatter matrix $\boldsymbol{S}$, sorted in order of decreasing corresponding eigenvalue
- Without any approximation, for any sample $\boldsymbol{x}_i$:

$$\boldsymbol{x}_i = \sum_{j=1}^{d} \alpha_j \boldsymbol{e}_j = \underbrace{\alpha_1 \boldsymbol{e}_1 + \ldots + \alpha_k \boldsymbol{e}_k}_{\textbf{approximation of } \boldsymbol{x}_i} + \overbrace{\alpha_{k+1} \boldsymbol{e}_{k+1} \ldots + \alpha_d \boldsymbol{e}_d}^{\textbf{error of approximation}}$$

- coefficients α_m are called *principle components*
  - The larger $\boldsymbol{k}$, the better is the approximation
  - Components are arranged in order of importance, more important components come first
- Thus PCA takes the first $\boldsymbol{k}$ most important components of $\boldsymbol{x}_i$ as an approximation to $\boldsymbol{x}_i$

PCA: Last Step

- Now we know how to project the data
- Last step is to change the coordinates to get final ***k***-dimensional vector ***y***



- Let matrix $\boldsymbol{E} = [\boldsymbol{e}_1 \cdots \boldsymbol{e}_k]$
- Then the coordinate transformation is $\boldsymbol{y} = \boldsymbol{E}^t \boldsymbol{x}$
- Under $\boldsymbol{E}^t$, the eigenvectors become the standard basis:

$$\boldsymbol{E}^t \boldsymbol{e}_i = \begin{bmatrix} \boldsymbol{e}_1 \\ \vdots \\ \boldsymbol{e}_i \\ \vdots \\ \boldsymbol{e}_k \end{bmatrix} \boldsymbol{e}_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

Recipe for Dimension Reduction with PCA

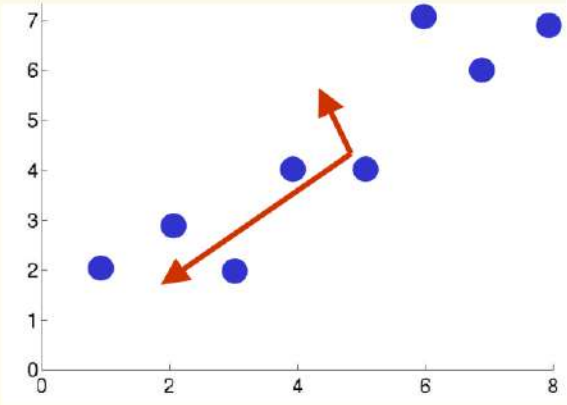
Data $\boldsymbol{D}=\{\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_n\}$. Each $\boldsymbol{x}_i$ is a $\boldsymbol{d}$-dimensional vector. Wish to use PCA to reduce dimension to $\boldsymbol{k}$

1. Find the sample mean $\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{x}_i$
2. Subtract sample mean from the data $\boldsymbol{z}_i = \boldsymbol{x}_i - \hat{\mu}$
3. Compute the scatter matrix $\boldsymbol{S} = \sum_{i=1}^{n} \boldsymbol{z}_i \boldsymbol{z}_i^t$
4. Compute eigenvectors $\boldsymbol{e}_1,\boldsymbol{e}_2,\ldots,\boldsymbol{e}_k$ corresponding to the $\boldsymbol{k}$ largest eigenvalues of $\boldsymbol{S}$
5. Let $\boldsymbol{e}_1,\boldsymbol{e}_2,\ldots,\boldsymbol{e}_k$ be the columns of matrix $\boldsymbol{E} = [\boldsymbol{e}_1 \cdots \boldsymbol{e}_k]$
6. The desired $\boldsymbol{y}$ which is the closest approximation to $\boldsymbol{x}$ is $\boldsymbol{y} = \boldsymbol{E}^t \boldsymbol{z}$

PCA Example Using Matlab

- Let $\boldsymbol{D} = \{(1,2),(2,3),(3,2),(4,4),(5,4),(6,7),(7,6),(9,7)\}$
- Convenient to arrange data in array

$$\boldsymbol{X} = \begin{bmatrix} 1 & 2 \\ \vdots & \vdots \\ 9 & 7 \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_8 \end{bmatrix}$$



- Mean $\mu = \boldsymbol{mean}(\boldsymbol{X}) = [4.6 \quad 4.4]$
- Subtract mean from data to get new data array Z

$$\boldsymbol{Z} = \boldsymbol{X} - \begin{bmatrix} \mu \\ \vdots \\ \mu \end{bmatrix} = \boldsymbol{X} - \boldsymbol{repmat}(\mu, 8, 1) = \begin{bmatrix} -3.6 & -4.4 \\ \vdots & \vdots \\ 4.4 & 2.6 \end{bmatrix}$$

- Compute the scatter matrix $\boldsymbol{S}$

$$\boldsymbol{S} = 7 * \boldsymbol{cov}(\boldsymbol{Z}) = [-3.6 \quad -4.4]\begin{bmatrix} -3.6 \\ -4.4 \end{bmatrix} + \ldots + [4.4 \quad 2.6]\begin{bmatrix} 4.4 \\ 2.6 \end{bmatrix} = \begin{bmatrix} 57 & 40 \\ 40 & 34 \end{bmatrix}$$

*matlab uses unbiased estimate for covariance, so S=(n-1)*cov(Z)*

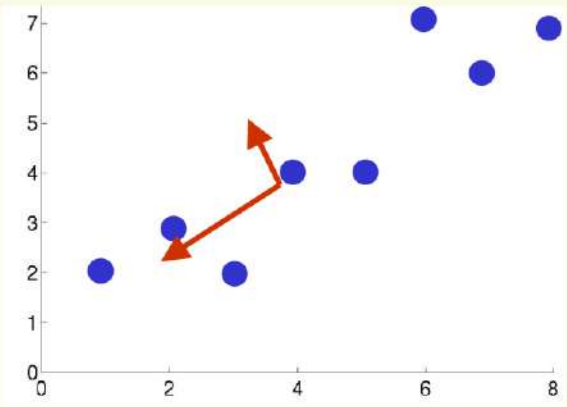
```python
import numpy as np

# Veri (yatay kompakt gösterim)
X = np.array([[1,2],[2,3],[3,2],[4,4],[5,4],[6,7],[7,6],[9,7]])

# Ortalama
mu = np.mean(X, axis=0)

# Centering
Z = X - mu

# Scatter matrix
S = np.cov(Z, rowvar=False) * (len(X) - 1)

print(mu)
print(S)
```

PCA Example Using Matlab

- Use [V,D] =eig(***S***) to get eigenvalues and eigenvectors of ***S***

$$\lambda_1 = 87 \text{ and } e_1 = \begin{bmatrix} -0.8 \\ -0.6 \end{bmatrix}$$

$$\lambda_2 = 3.8 \text{ and } e_2 = \begin{bmatrix} 0.6 \\ -0.8 \end{bmatrix}$$

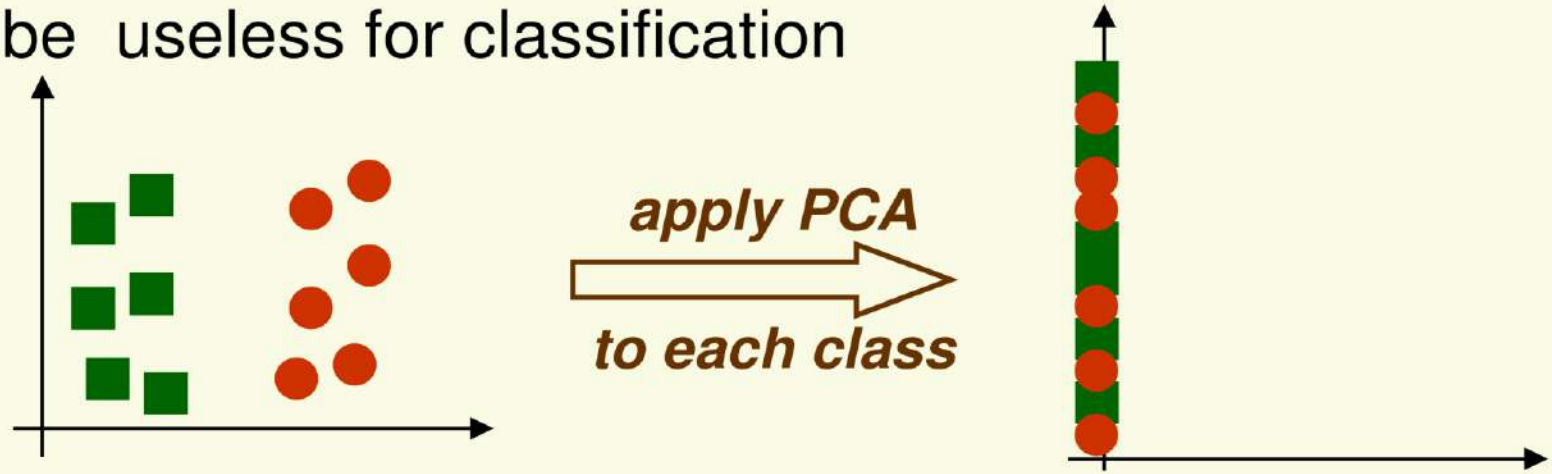


- Projection to 1D space in the direction of e_1

$$Y = e_1^t Z^t = \left(\begin{bmatrix} -0.8 & -0.6 \end{bmatrix} \begin{bmatrix} -3.6 & \cdots & 4.4 \\ -4.4 & \cdots & 2.6 \end{bmatrix} \right) = \begin{bmatrix} 4.3 & \cdots & -5.1 \end{bmatrix}$$

$$= \begin{bmatrix} y_1 & \cdots & y_8 \end{bmatrix}$$

```python
# 3. Scatter matrix
S = np.cov(Z, rowvar=False) * (len(X) - 1)

# 4. Özdeğer ve özvektörler
eigvals, eigvecs = np.linalg.eig(S)

# 5. En büyük özdeğeri bul ve karşılık gelen özvektörü seç
idx = np.argmax(eigvals)
e1 = eigvecs[:, idx]

# 6. 1D projeksiyon
Y = Z @ e1   # (alternatif: e1.T @ Z.T)

print("Eigenvalues:\n", eigvals)
print("\nPrincipal eigenvector (e1):\n", e1)
print("\nProjected data (Y):\n", Y)
```

Drawbacks of PCA

- PCA was designed for accurate *data representation*, not for data classification
  - Preserves as much variance in data as possible
  - If directions of maximum variance is important for classification, will work
- However the directions of maximum variance may be useless for classification